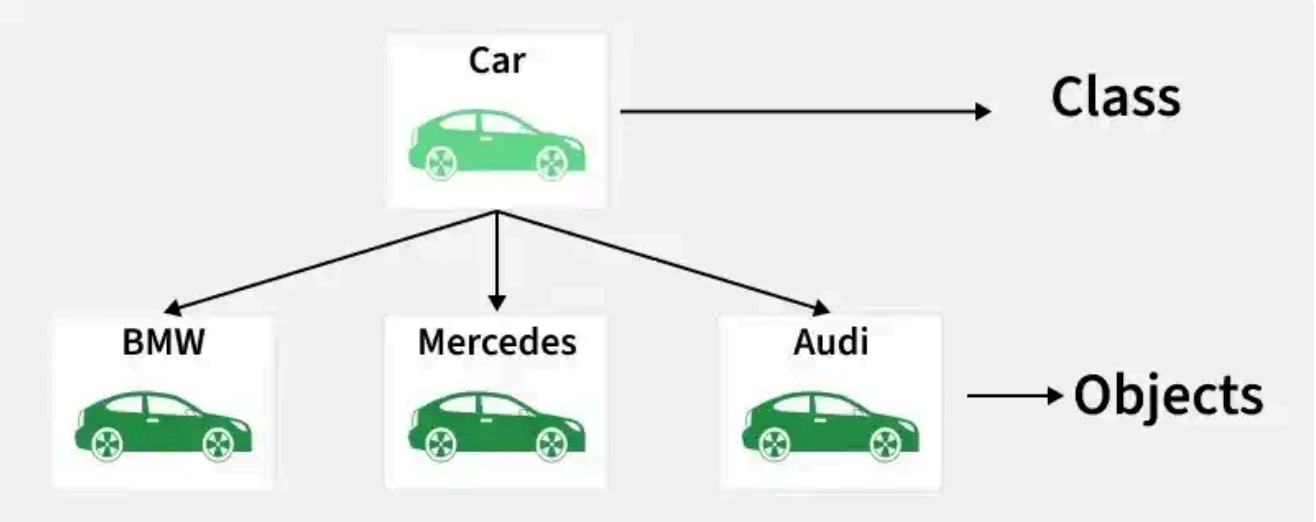
OOP Intro

Object-Oriented Programming (OOP) organizes software around objects rather than functions and logic. An object contains both data (attributes) and methods (behavior), making programs more modular and reusable. OOP helps developers build scalable, maintainable, and real-world applications efficiently.

Class

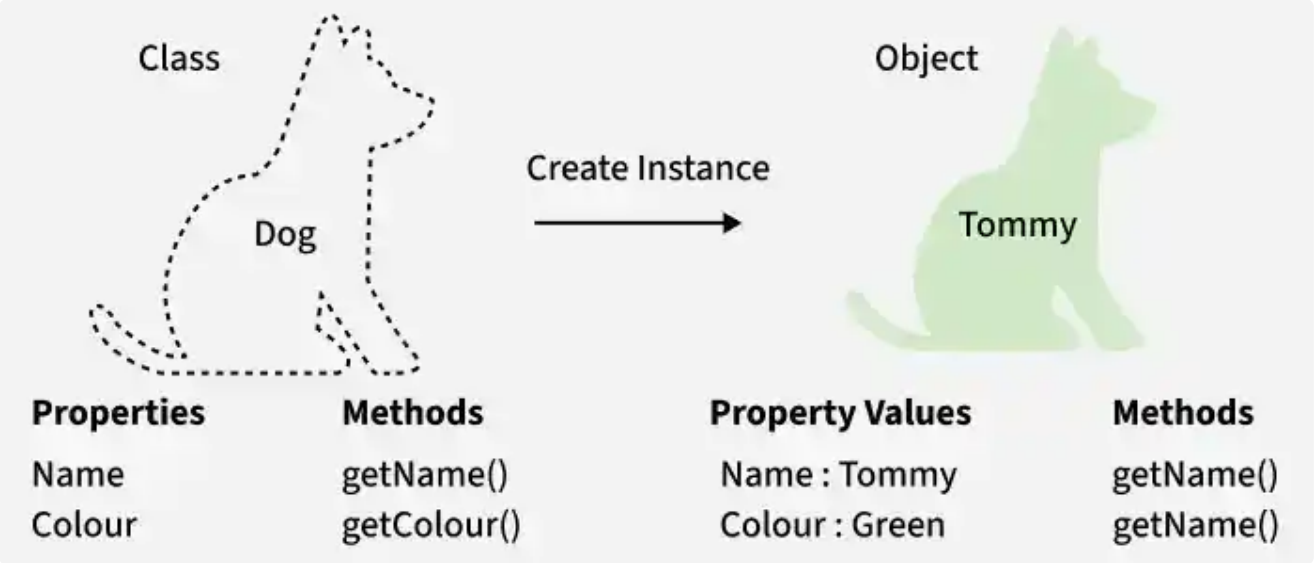
Class is like a blueprint. Let's say we create a class Animals, now this blueprint defines a few pieces of data that every animal must have (eg: name, age). These pieces of data are called attributes/properties.
We can also define actions that each animal performs, (eg: speak) these actions are called methods.
Now, every object created of Class Animals must follow all the rules of it, it should have all the data and should be capable of performing the methods
The reason i say Class is like a blueprint is that it will not occupy any memory until an Object with properties is created. The blueprint of a machine does not really take any space, but once the machine is built it will take a lot of space.



Object

An object is basically an example of a class(instance). An item that belongs to that class and holds the properties of that class.
An item built from the blueprint.
Each object has their own copy of data, which do not interfere with each other.
We can perform actions on the object using the methods we defined in the class.

We can also access the attributes of the object by using the attributes name defined in class



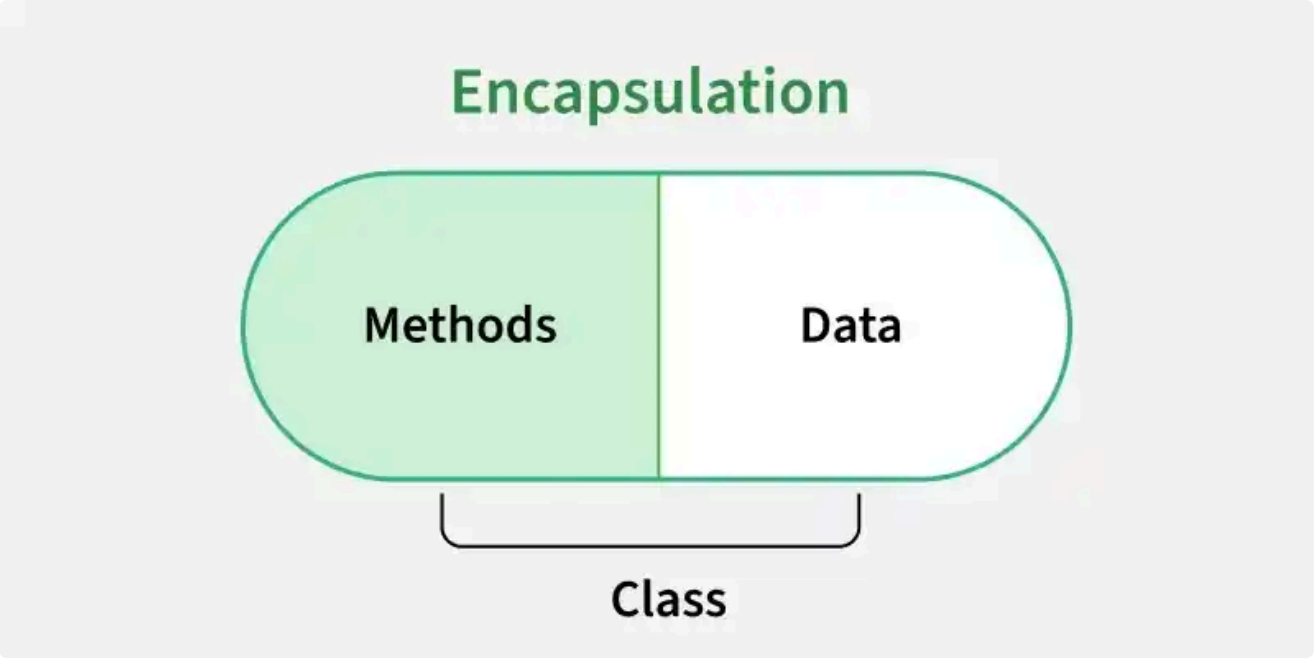
Encapsulation

Encapsulation is the process of binding data and methods together into a single unit and restricting direct access to data using access modifiers.
It revolves around two main ideas:

- **Bundling 💊:** Keeping the data (attributes) and the actions that use that data (methods) safely together in one place (the class).
- **Protection 🛡️:** Hiding the internal state of an object from the outside world and restricting direct access to it.

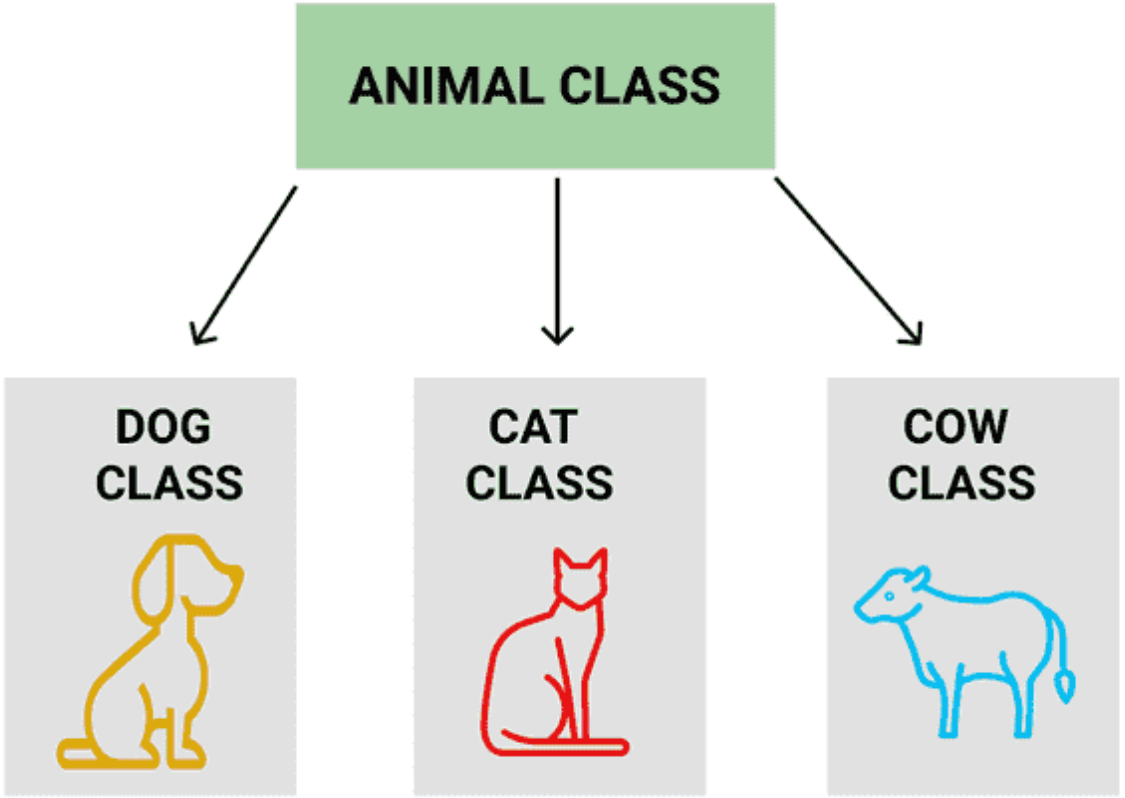
We do this to prevent other parts of our program from accidentally breaking the rules.
Let's look at a `Player` class. If `health` is unprotected, someone could write this in the code: `player1.health = -5000`
Having negative health doesn't make sense in a game! To fix this in Python, we encapsulate data by adding two underscores `__` in front of the attribute name to make it "private."
Because of the `__`, if you try to forcefully run `player1.__health = -5000` from outside the class now, Python will block it.

Encapsulation makes us access data with getter and setter methods, which is much safer than directly accessing it.



Inheritance

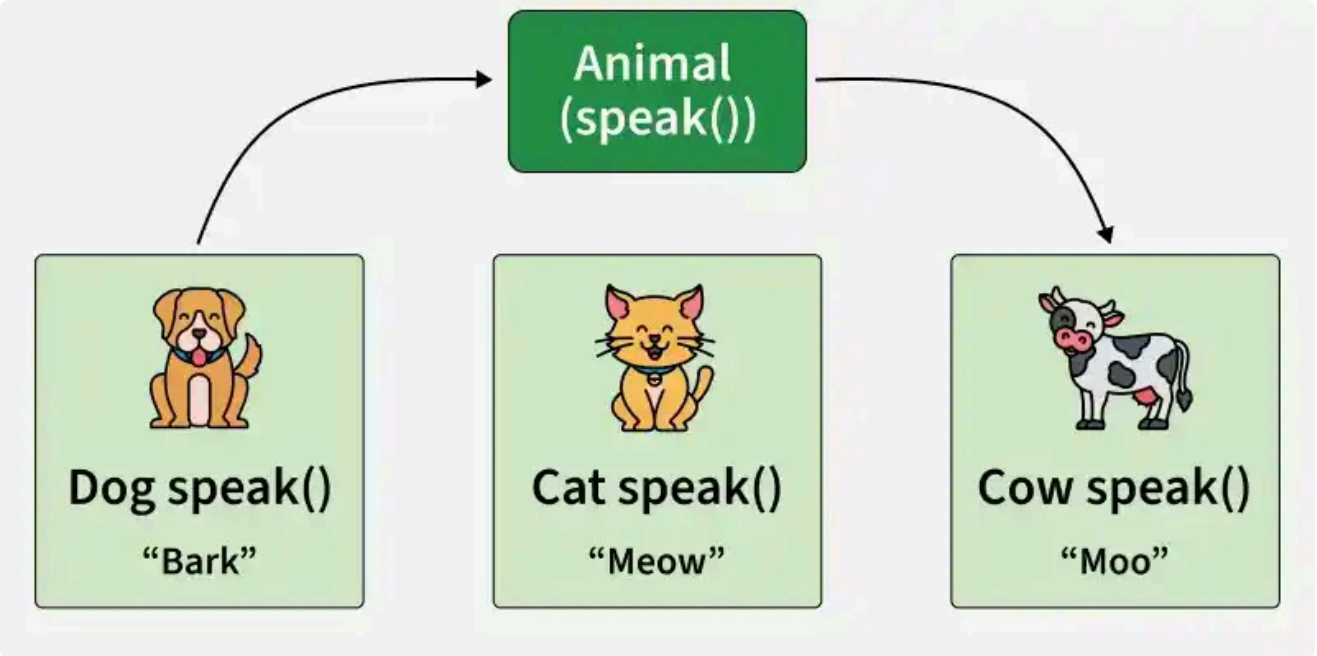
Inheritance allows one class to acquire the properties and methods of another class.
All objects within Animal class have data, and this data can be passed to a subclass, say, Dog, we can add extra properties in Dog subclass, (eg: breed).



Polymorphism

Polymorphism means "many forms" and allows the same method or interface to perform different actions depending on the object that invokes it. It improves flexibility and enables dynamic behavior in object-oriented applications.

Let's say, class Animals has a method speak(), however all animal don't produce the same sound, so we make a default speak() in the super class, and we overwrite in the subclass, say, Dog with speak(bark).



Advantages of OOP

- ***Modularity:*** Programs are divided into independent classes, making development easier.
- ***Easy Maintenance:*** Changes in one part of the program have minimal impact on others.
- ***Improved Readability:*** Well-structured classes make code easier to understand.
- ***Scalability:*** Applications can be expanded and enhanced without major modifications.
- ***Faster Development:*** Reusable components and modular design speed up development.
- ***Better Testing and Debugging:*** Individual classes can be tested and debugged independently.
- ***Real-World Modeling:*** Objects represent real-world entities, making system design intuitive.

Limitations of OOP

While OOP provides many benefits, it also has some limitations:

- ***Steeper Learning Curve:*** Concepts such as classes, objects, inheritance, polymorphism, abstraction, and encapsulation can be challenging for beginners.
- ***Additional Overhead for Small Programs:*** OOP may introduce extra classes and structure that are unnecessary for simple applications.
- ***Increased Design Complexity:*** Designing a proper class hierarchy and object relationships requires careful planning.

- ***Higher Memory Consumption:*** Creating and managing a large number of objects can require more memory compared to procedural approaches.